# Robust In-Hand Reorientation With Hierarchical RL-Based Motion Primitives and Model-Based Regrasping

Yongpeng Jiang, Mingrui Yu, *Graduate Student Member, IEEE*, Chen Chen, *Graduate Student Member, IEEE*, Yongyi Jia, and Xiang Li, *Senior Member, IEEE*

***Abstract*—In-hand manipulation has become increasingly popular in recent robotics research, probably due to the growing trend toward humanoids and artificial general intelligence. Although existing works show promising results, they typically require expensive dexterous hands, depth cameras, and tactile sensors, which are challenging for algorithm reproduction and large-scale applications. To address this issue, this article proposes a practical and low-cost solution to the classic in-hand cube reorientation task. The proposed method is characterized by a hierarchical structure. Specifically, several in-hand motion primitives, such as object rotation and flipping, are trained using reinforcement learning. A high-level decision module switches between these motion primitives, adapting to specific task requirements without the need for retraining. We demonstrate that the proposed method achieves continuous reorientation of the cube for nearly 20 min using only a low-cost LEAP Hand and a single RGB camera. The proposed method was the winning solution for in-hand manipulation at the 9th Robotic Grasping and Manipulation Competition during ICRA 2024. Implementation details and evaluation results are discussed in this article. We also open-source hardware designs, code, and videos, which are available online, to encourage reimplementation and further development.**

***Index Terms*—In-hand manipulation, reinforcement learning (RL), robotic grasping and manipulation competition (RGMC).**

## I. Introduction

DEXTEROUS in-hand manipulation is a fundamental skill in humanoid research nowadays [1], [2]. Such a skill achieves human-level dexterity with minimal space and energy consumption, compared to arm-hand coordination [3]. However, real-world deployment of in-hand manipulation is often corrupted by noise and disturbances. For example, manipulation with making and breaking contacts exhibits more dynamic motions and larger modeling errors compared to classical grasping-based manipulation [4], [5]. The problems are more severe with low-cost hardware and limited sensing modalities, which are common in real-world applications [6].

In this article, we consider the task of robotic in-hand reorientation, which is a well-studied task in many existing works. Specifically, the goal is to manipulate a cube in order to reach successive targets. The in-hand cube reorientation task has been benchmarked by the 9th Robotic Grasping and Manipulation Competition (RGMC) at ICRA 2024 [7], [8], where the goals are defined by a sequence of target faces, and a goal is reached when the normal of the target face is aligned with the camera's optical axis within a given threshold.

Addressing the task of in-hand reorientation is not trivial. On the one hand, the reorientation task is long-horizon, thus increasing the probability of failure (e.g., the object falls). The large object displacement requires the fingers to constantly change contact locations, leading to unstable state transitions. On the other hand, it is not trivial to coordinate different fingers. Once the force balance is broken, the object may be pushed out of the hand. This is likely to happen if unexpected contacts are made, or if some planned contacts are missed due to slippage, which is common in multifingered manipulation.

Among existing works on in-hand reorientation, reinforcement learning (RL)-based approaches achieve state-of-the-art performance through large-scale parallel training and domain randomization [9], [10], [11]. However, RL policies struggle to learn long-horizon tasks, such as in-hand reorientation from scratch [12]. In addition, many existing approaches use commercial dexterous hands and advanced sensing, but performance on low-cost hardware remains uncertain [13]. Apart from RL approaches, model-based control [4], [5], [14] and predictive sampling [15] also show promise for in-hand reorientation. However, these methods cannot achieve the highly dynamic manipulation seen in RL policies due to their high online computational costs. To achieve robust and dexterous in-hand reorientation with low-cost hardware, this article proposes a hierarchical framework, as shown in Fig. 1. Specifically, the execution of in-hand reorientation is divided into a sequence of motion primitives, including rotation and flipping. We train a proprioception-only RL policy for each of the motion primitives. At a high level, we

Received 8 March 2025; accepted 6 November 2025. Date of publication 17 November 2025; date of current version 2 January 2026. This work was supported in part by the Science and Technology Innovation 2030-Key Project under Grant 2021ZD0201404, in part by the Beijing National Research Center for Information Science and Technology and Tsinghua University Initiative Scientific Research Program, and in part by the National Natural Science Foundation of China under Grant U21A20517, Grant 62461160307, and Grant 623B2059. This article was recommended for publication by Associate Editor Peter So upon evaluation of the reviewers' comments. *(Corresponding author: Xiang Li.)*

The authors are with the Department of Automation, Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: xiangli@tsinghua.edu.cn).

Data is available online at https://rgmc-xl-team.github.io/inhand_reorientation/.

This article has supplementary downloadable material available at https://doi.org/10.1109/RAP.2025.3633232, provided by the authors.

Digital Object Identifier 10.1109/RAP.2025.3633232

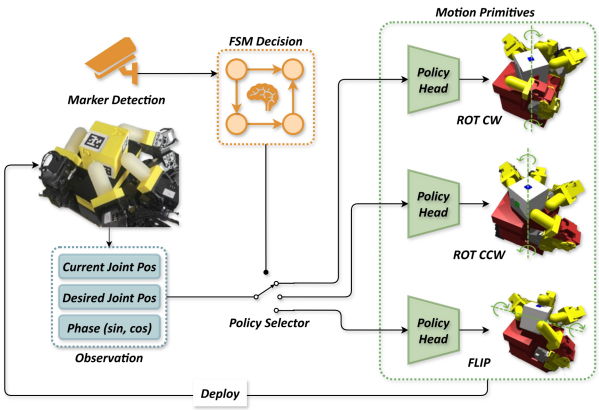



Fig. 1. Overview of the proposed framework. The task is described by a sequence of target cube faces. The proposed framework has a hierarchical structure. The FSM decision module relies on marker-based detection to select the RL-based motion primitives. The RL policies are trained and deployed with proprioceptive inputs only.

design a finite state machine (FSM) to schedule the execution of the motion primitives. To ensure that the initial state is in the best performance region of the training distribution, we propose a model-based regrasping strategy to reposition the object to the center of the palm after the execution of each motion primitive. Compared to end-to-end approaches, the proposed method exhibits superior robustness by hierarchically decomposing long-horizon tasks, which ensures thorough exploration of subgoals and avoids the cumulative errors associated with directly training on long-horizon tasks. In addition, the proposed method enhances modularity by reusing trained motion primitives. Thus, the high-level strategy can be adjusted according to task requirements without the need to retrain the entire policy network.

Contributions of the article can be summarized as follows.

1) We propose a hierarchical framework combining RL-based motion primitives and model-based regrasping strategy, enabling robust in-hand cube reorientation with low-cost hardware. Due to its satisfactory robustness, our approach has won the championship of the in-hand manipulation track of the RGMC 2024.
2) Beyond the competition results, we conducted additional experiments to thoroughly evaluate the proposed method's performance. We open-source our hardware designs, code, and videos in the Supplementary Material to facilitate reimplementation for practitioners.

## II. Related Works

### A. In-Hand Manipulation With Multifingered Hands

The tasks of multifingered in-hand manipulation can be divided into two categories [16]. The first category, also known as in-grasp manipulation, involves moving the object in the hand without making and breaking contacts [3], [17], [18]. However, the movement of the object is restricted to a short range due to the limited reachable space of the fingers and the lack of regrasping. The second category involves frequent contact-making and breaking between the object and the fingers [4], [5], [19], [20], which is the problem studied in this article. The object can be repositioned to achieve any SE(3) target pose within the hand's workspace. However, the discrete contact events contribute a large part of the instability, since contact states are mutable and slippage can occur under possible perturbations [21]. Existing works use both learning-based and model-based methods to solve multifingered in-hand manipulation. Recent developments in contact modeling make it tractable to deploy model-based control algorithms on hardware systems [22], [23], [24]. However, due to inevitable modeling errors and high online computational requirements, model-based control has only been successfully demonstrated on relatively simple and quasi-dynamic in-hand manipulation tasks [4], [20]. In contrast, RL shows surprising robustness on the dynamic, dexterous in-hand manipulation tasks [25], [26]. The superior real-world performance of RL methods benefits from large-scale training, multimodal sensory fusion, and sim-to-real techniques [13].

### B. In-Hand Reorientation With RL

Solving the in-hand reorientation task with RL has been a long-standing challenge, and efforts have been made to improve robustness, dexterity, and the use of tactile information [11], [18], [27]. OpenAI demonstrated successful real-world cube reorientation using an RL-based controller and a vision-based pose estimator trained in simulation [13]. The hardware setup includes tracking cameras and the expensive shadow hand. Chen et al. [9] later proposed a two-stage teacher–student training procedure and achieved general in-hand reorientation with a single depth camera and a low-cost dexterous hand. More recently, the authors in [10] and [28] achieved general in-hand rotation with multimodal perception and rapid motor adaptation. Their strategy achieves single-axis rotation after only several hours of training, but the object falls out of the hand within a minute due to disturbance. The aforementioned works follow the end-to-end fashion, which has low data efficiency in learning the long-horizon manipulation and faces performance degradation during long-term execution. The robotics community has explored hierarchical strategies to decompose the complexity into different levels [12]. Successful attempts have been made to solve the in-hand manipulation problem, including hierarchical model-based planning [29] and hierarchical RL [30], [31], [32]. A recent work developed a hierarchical RL approach for in-hand reorientation based on previously acquired low-level skills [33]. Compared to their approach, our high-level strategy is deterministic rather than RL-based, offering greater robustness and easier adaptation to new task requirements in practice.

## III. Methods

### A. RL-Based Motion Primitives

We formulate the low-level tasks, such as object rotation and flipping, as finite-horizon Markov decision processes. The design of the observation, action space, and rewards will be discussed in this section.

*1) Observation:* In order to minimize the hardware cost, the proposed method does not require tactile sensors. Thus,

the observation $\boldsymbol{o}_t$ contains only proprioception information, including the current and desired joint position $\boldsymbol{q}_t, \boldsymbol{q}_t^d \in \mathbb{R}^{16}$. Similar to [28] and [34], to facilitate efficient learning of periodic motions, the phase variables $\sin(\tau_t), \cos(\tau_t) \in \mathbb{R}$ are appended to the observations, where $\tau_t$ is the wall clock time of step $t$ since policy deployment starts. Three history frames $\{\boldsymbol{o}_{t-2}, \boldsymbol{o}_{t-1}, \boldsymbol{o}_t\}$ are concatenated as the final observation $\mathcal{O}_t$.

*2) Action Space:* The action $\boldsymbol{a}_t \triangleq \Delta \boldsymbol{q}_t^d \in \mathbb{R}^{16}$ is the delta desired joint positions of the joint actuators. The action is bounded by $\boldsymbol{a}_t \in [-0.042, 0.042]^{16}$rad.

*3) Reward Design:* We design different reward functions for the rotation and flipping motion primitives. As illustrated in Fig. 1, the rotation task [i.e., rotation clockwise policy (ROT CW)/counter clockwise policy (CCW)] involves continuously holding and rotating the cube around the axis perpendicular to the palm, guided by the following reward:

$$r_{\text{rot}} = r_{\text{angvel}} + r_{\text{linvel}} + r_{\text{dof}} + r_{\text{energy}} + r_{\text{torque}} + r_{\text{pen}}. \quad (1)$$

Among the reward terms, $r_{\text{angvel}}$ encourages continuous rotation, $r_{\text{linvel}}$ penalizes large object velocities, $r_{\text{dof}}$ penalizes the joint angle difference from initial grasping, and $r_{\text{energy}}, r_{\text{torque}}$ penalizes energy consumption and excessive joint torques. Besides, $r_{\text{pen}}$ is a large penalty that takes effect when the object moves out of the palm center, which is added to produce more stable motions. In contrast, the flipping task requires the block to be flipped one face at a time on the palm, achieving a rotation of approximately 90°each time. This is trained using a goal-conditioned reward

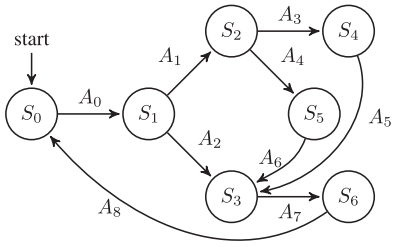
$$r_{\text{flip}} = r_{\text{ftip}} + r_{\text{contact}} + r_{\text{goal}} + r_{\text{angvel}} + r_{\text{dof}} + r_{\text{energy}} + r_{\text{torque}} + r_{\text{pen}}. \quad (2)$$

Note that three additional reward terms are added for the flipping task: $r_{\text{ftip}}$ encourages all fingers to make contact, securing the object and reducing reliance on external dexterity; $r_{\text{contact}}$ penalizes contact with slippery areas to prevent instability; $r_{\text{goal}}$ promotes reaching the goal. The mathematical forms and weighting coefficient for each reward term can be found in the Supplementary Material.

*4) Policy Training:* We use 8192 parallel environments in Isaac Gym [35] to collect the interaction data and use proximal policy optimization [36] to optimize the policy. For sim-to-real, we apply domain randomization on the object properties (i.e., mass, scale, and friction) and robot properties [i.e., friction and proportional-derivative (PD) gains], which are discussed in detail in the Appendix. Similar to Hora [10], we generate a cache of 1024 grasping poses as the reset distribution, which greatly promotes exploration.

### B. High-Level Decision Module

*1) FSM Design:* The high-level decision module is designed as an FSM (see Fig. 2). When switching to the next target face, since we only successfully deploy the flipping policy (FLIP) policy with the direction in Fig. 1 on hardware due to the mechanical structure of the LEAP Hand, the ROT CW/CCW policies are deployed first if the target face is unreachable with one flip.




| node | state |
|---|---|
| $S_0$ | INIT |
| $S_1$ | WAIT |
| $S_2$ | BEFORE_ROT |
| $S_3$ | BEFORE_FLIP |
| $S_4$ | ROT_CW |
| $S_5$ | ROT_CCW |
| $S_6$ | FLIP |

| edge | condition | action |
|---|---|---|
| $A_0$ | | reset |
| $A_1$ | target face reachable after rotation | |
| $A_2$ | target face reachable via one flip | |
| $A_3$ | need CW rotation | deploy ROT CW policy |
| $A_4$ | need CCW rotation | deploy ROT CCW policy |
| $A_5$ | target face reachable via one flip | stop current policy |
| $A_6$ | target face reachable via one flip | stop current policy |
| $A_7$ | | deploy FLIP policy |
| $A_8$ | target face reached | |

Fig. 2. High-level FSM design.

*2) Object Detection:* The cube's pose is detected and filtered based on the AprilTags attached on all six faces. Since visual information is not observed by the RL policies, the occlusion has ignorable effects on the motion primitives. The pose information is only needed for the object relocalization process and terminating the running policy.

*3) Object Relocalization:* We design a regrasping strategy to relocalize the cube after the deployment of each policy. This guarantees that the object is approximately placed at the palm center and improves the success rate of the subsequent motion primitive. We first design an inverse kinematics (IK) algorithm, which solves the following optimization problem:

$$\min_{\boldsymbol{q}} \sum_{i=1}^{4} \|\boldsymbol{p}_i - \boldsymbol{p}_{i,\text{tgt}}\|_{\boldsymbol{W}_\text{p}} + \|\ln\left(\boldsymbol{R}_i \boldsymbol{R}_{i,\text{tgt}}^\top\right)^\vee\|_{\boldsymbol{W}_\text{r}} \quad (3)$$

where $\boldsymbol{p}$ and $\boldsymbol{R}$ are the configuration-dependent position and rotation matrix of the fingertips, $\boldsymbol{p}_{\text{tgt}}$ and $\boldsymbol{R}_{\text{tgt}}$ are the target values, $\ln(\cdot)^\vee$ is the transformation from the rotation matrix to the axis–angle rotation vector, and $\boldsymbol{W}_\text{p}$ and $\boldsymbol{W}_\text{r}$ are the weighting matrices of the translation and rotation parts. To achieve the regrasping, we first compute the desired contact points by projecting each fingertip onto the cube, and then, grasp the cube by solving (3) that aligns the fingertips with the contact points. The contact points move with the object, which is assumed to be repositioned to the center of the palm, the fingers then track the contact points. We solve the optimization problem (3) with SciPy in the code.

## IV. Experiments

### A. Experimental Setup

We replace the original fingertips of LEAP Hand [34] with 3D-printed polylactic acid (PLA) bones and silicone sleeves, and

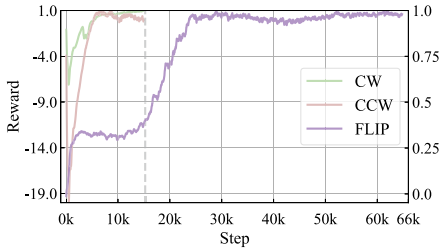



Fig. 3. Normalized reward in the training of motion primitives. The vertical axis of ROT CW/CCW is on the left. The vertical axis of FLIP is on the right.

mount the hand on a stationary frame. The palm is heightened to ensure that the cube is within the reachable space of the fingers. We also mount a RealSense D405 camera above the hand, and use RGB images only. The proposed method is implemented with robot operating system (ROS) [37] on a laptop with AMD Ryzen 7 5800H CPU and NVIDIA RTX 3060 GPU. More details about the hardware setup and the proposed method can be found in the Supplementary Material.

### B. RL-Based Training of Motion Primitives

The RL policies are trained on a single NVIDIA RTX 3090 GPU, and the training reward is shown in Fig. 3. Note that the reward is normalized by the maximum positive value. The results show that it takes 3–4 times longer to train the FLIP policy, since the cube needs to be pivoted on an edge and the maximum reached angle increases gradually. While the ROT CW/CCW policies simply need to learn the finger gaiting through a wrench-balanced grasp, which is much easier. The training of ROT and FLIP policies takes around 5.5 and 25 h, respectively. The RL-based motion primitives are deployed at 20 Hz. The average rotation speeds of the ROT CW, ROT CCW, and FLIP policies are 0.68, 0.5, and 0.69 rad/s, respectively. The ROT policies achieve continuous rotation for at least 1 min and the FLIP policy fails after 5.8 successful flips on average. Real-world deployments with external disturbances are shown in the Supplementary Material.

### C. Model-Based Regrasping Strategy

To quantitatively evaluate the proposed regrasping strategy, we manually change the position and orientation of the cube and execute regrasping. The above process is repeated 100 times, and the statistical results are shown in Fig. 4. Using the data collected from the complete task in Section IV-D, we estimate the distribution of the cube's center of mass (COM) position immediately after the execution of motion primitives. The distribution is indicated by the background color. The results show that the distribution of the cube's COM position is more concentrated after regrasping, and almost all the points in the high-density region migrate near the palm center at $[-0.05\ \text{m}, 0.03\ \text{m}]$. Note that several data points fall into the top left corner, which lies on the boundary of the fingers' reachable space. For these points, we execute an effective two-stage strategy, which moves the cube approximately back to the palm center using the thumb

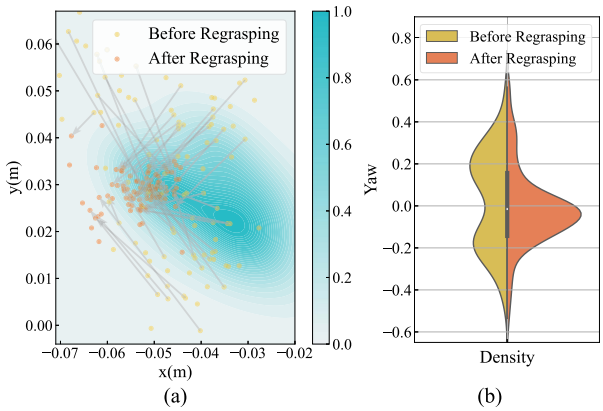



Fig. 4. (a) Distribution of the cube's COM position on the *XY* plane. The grey arrow represents the cube's displacement due to regrasping. The blue background indicates the probability density of cube's COM position observed immediately after the execution of motion primitives. (b) Distribution of the cube's yaw angle before and after regrasping.

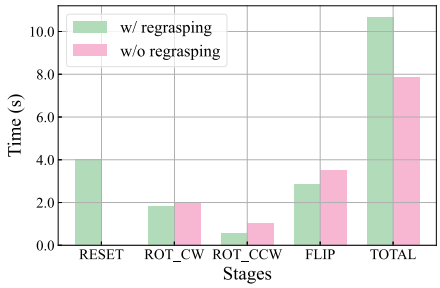



Fig. 5. Comparison of the time consumed in different stages.

and ring finger, and then apply the regrasping to adjust the cube's position. Due to the limited workspace of fingers, some data points are not effectively repositioned to the palm center, which has a negligible effect on the complete task since these points stay in the low-density region. Besides, the cube's orientation is centered around $0.0\ \text{rad}$ after regrasping. The results reveal that the proposed regrasping strategy reduces the cube's pose uncertainty and ensures that the observation is more "in distribution" for the subsequent RL-based motion primitives.

### D. In-Hand Cube Reorientation

To further validate the proposed framework and the regrasping strategy, we generate a sequence containing 100 target faces. The elapsed time for each stage is visualized in Fig. 5. In three repeated experiments, the proposed framework successfully executes all 100 faces, while the ablation without regrasping fails after 10 faces on average. The time consumed for RL-based motion primitives significantly increases without regrasping, as indicated in Fig. 5. This is because the misaligned cube leads to out-of-distribution data and the system takes longer to realize stable finger gaiting, which can be better visualized in the Supplementary Video. It takes about 10.6 s for the proposed method to perform one reorientation (i.e., switch to a next target face). The proposed method significantly improves the success

TABLE I
STATEMENT OF SUPPLEMENTARY MATERIALS

| Materials | Description |
|---|---|
| Source code | We open source the code for reproducing our competition results. We recommend the readers to access and use the code via GitHub. We provide a detailed guideline for setting up everything and to launch the experiments. Users are encouraged to report issues via GitHub. We believe that this is an effective way of sharing the code even without the "Reproducible Run" process on Code Ocean. The readers can directly access the GitHub repository via this link. |
| Dataset | The trained RL checkpoints and the pre-generated grasp cache for training the RL policies are provided. The readers can download the datasets and follow the guidelines in the .txt files to prepare these datasets. The dataset has been uploaded to Google Drive and can be accessed through the project website. |
| Video | We provide video of the real-world experiment results. The video can be viewed online through the project website |
| Computer aided design (CAD) files | We provide CAD files of the hand mount and our customized fingertips. The CAD files have been uploaded to Google Drive and can be accessed through the project website. |
| Appendix | Due to the page limits, we provide some important details in the Appendix. The details include hyper-parameters and reward design of the RL training, as well as snapshots of the hardware experiments. The Appendix can be accessed through the project website. |

rate at the cost of slightly increasing the execution time. Besides, the end-to-end policy that reorients the object given an arbitrary goal pose even fails in the simulation, highlighting the necessity of a hierarchical policy.

With regard to the in-hand manipulation (RGMC), comparison results are available on the Official Website regarding metrics "Waypoints Reached" and "Runtime." Notably, the winner of RGMC 2025 continued to use the proposed method, as demonstrated in the Supplementary Material.

## V. CONCLUSION

This article proposes a hierarchical framework to accomplish the task of in-hand cube reorientation, which won the championship of the RGMC in-hand manipulation track at ICRA 2024. The framework leverages RL-based motion primitives and the model-based regrasping strategy to achieve robust real-world performance within several hours of training. Experiments show that our approach achieves nearly 20 min of continuous manipulation. Furthermore, the regrasping strategy significantly reduces object position uncertainty and improves the stability of deploying RL policies. Beyond providing a detailed solution, this article aims to provide insights into real-world robustness gained from the competition.

## APPENDIX

The supplementary materials can be easily accessed from the project website. The type of material along with a brief description are listed in Table. I.

## REFERENCES

[1] A. Billard and D. Kragic, "Trends and challenges in robot manipulation," *Science*, vol. 364, no. 6446, 2019, Art. no. eaat8414.
[2] S. Cruciani, B. Sundaralingam, K. Hang, V. Kumar, T. Hermans, and D. Kragic, "Benchmarking in-hand manipulation," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 588–595, Apr. 2020.
[3] M. Yu, Y. Jiang, C. Chen, Y. Jia, and X. Li, "Robotic in-hand manipulation for large-range precise object movement: The RGMC champion solution," *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, pp. 4738–4745, May 2025.
[4] T. Pang, H. T. Suh, L. Yang, and R. Tedrake, "Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models," *IEEE Trans. Robot.*, vol. 39, no. 6, pp. 4691–4711, Dec. 2023.
[5] Y. Jiang, M. Yu, X. Zhu, M. Tomizuka, and X. Li, "Contact-implicit model predictive control for dexterous in-hand manipulation: A long-horizon and robust approach," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2024, pp. 5260–5266.
[6] M. Yu et al., "In-hand following of deformable linear objects using dexterous fingers with tactile sensing," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2024, pp. 13518–13524.
[7] "9th robotic grasping and manipulation competition," (n.d.). [Online]. Available: https://cse.usf.edu/~yusun/rgmc/2024.html
[8] "Essential skill sub-track 2: In-hand manipulation," (n.d.). [Online]. Available: https://hangkaiyu.github.io/RGMC_in_hand_manipulation_subtrack.html
[9] T. Chen, M. Tippur, S. Wu, V. Kumar, E. Adelson, and P. Agrawal, "Visual dexterity: In-hand reorientation of novel and complex object shapes," *Sci. Robot.*, vol. 8, no. 84, 2023, Art. no. eadc9244.
[10] H. Qi et al., "General in-hand object rotation with vision and touch," in *Proc. Conf. Robot Learn.*, 2023, pp. 2549–2564.
[11] L. Sievers, J. Pitz, and B. Bäuml, "Learning purely tactile in-hand manipulation with a torque-controlled hand," in *Proc. Int. Conf. Robot. Autom.*, 2022, pp. 2745–2751.
[12] S. Pateria, B. Subagdja, A.-H. Tan, and C. Quek, "Hierarchical reinforcement learning: A comprehensive survey," *ACM Comput. Surv.*, vol. 54, no. 5, 2021, Art. no. 109.
[13] O. M. Andrychowicz et al., "Learning dexterous in-hand manipulation," *Int. J. Robot. Res.*, vol. 39, no. 1, pp. 3–20, 2020.
[14] Y. Jiang, M. Yu, X. Zhu, M. Tomizuka, and X. Li, "Robust model-based in-hand manipulation with integrated real-time motion-contact planning and tracking," 2025, *arXiv:2505.04978*.
[15] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa, "Predictive sampling: Real-time behaviour synthesis with mujoco," 2022, *arXiv:2212.00541*.
[16] R. R. Ma and A. M. Dollar, "On dexterity and dexterous manipulation," in *Proc. 15th Int. Conf. Adv. Robot.*, 2011, pp. 1–7.
[17] B. Sundaralingam and T. Hermans, "Relaxed-rigidity constraints: Kinematic trajectory optimization and collision avoidance for in-grasp manipulation," *Auton. Robots*, vol. 43, pp. 469–483, 2019.
[18] M. Yang et al., "Anyrotate: Gravity-invariant in-hand object rotation with sim-to-real touch," in *Proc. Conf. Robot Learn.*, 2024, pp. 4727–4747.
[19] W. Jin, "Complementarity-free multi-contact modeling and optimization for dexterous manipulation," in *Proc. Robot.: Sci. Syst. XXI*, 2025.
[20] W. Yang and W. Jin, "ContactSDF: Signed distance functions as multi-contact models for dexterous manipulation," *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, pp. 4212–4219, May 2025.
[21] A. Lakshmipathy and N. S. Pollard, "ContactMPC: Towards online adaptive control for contact-rich dexterous manipulation," in *Proc. 2nd Workshop Dexterous Manipulation: Des., Perception Control*, 2024. [Online]. Available: https://openreview.net/forum?id=d8qYLDH2vj
[22] W. Jin and M. Posa, "Task-driven hybrid model reduction for dexterous manipulation," *IEEE Trans. Robot.*, vol. 40, pp. 1774–1794, 2024.
[23] R. S. Zarrin, R. Jitosho, and K. Yamane, "Hybrid learning-and model-based planning and control of in-hand manipulation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 8720–8726.
[24] A. Nagabandi, K. Konolige, S. Levine, and V. Kumar, "Deep dynamics models for learning dexterous manipulation," in *Proc. Conf. Robot Learn.*, 2020, pp. 1101–1112.
[25] J. Pitz, L. Röstel, L. Sievers, and B. Bäuml, "Dextrous tactile in-hand manipulation using a modular reinforcement learning architecture," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 1852–1858.
[26] C. Yu and P. Wang, "Dexterous manipulation for multi-fingered robotic hands with reinforcement learning: A review," *Front. Neurorobot.*, vol. 16, 2022, Art. no. 861825.
[27] A. Handa et al., "DeXtreme: Transfer of agile in-hand manipulation from simulation to reality," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 5977–5984.
[28] H. Qi, A. Kumar, R. Calandra, Y. Ma, and J. Malik, "In-hand object rotation via rapid motor adaptation," in *Proc. Conf. Robot Learn.*, 2023, pp. 1722–1732.

[29] X. Cheng, S. Patil, Z. Temel, O. Kroemer, and M. T. Mason, "Enhancing dexterity in robotic manipulation via hierarchical contact exploration," *IEEE Robot. Autom. Lett.*, vol. 9, no. 1, pp. 390–397, Jan. 2024.

[30] T. Li, K. Srinivasan, M. Q.-H. Meng, W. Yuan, and J. Bohg, "Learning hierarchical control for robust in-hand manipulation," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2020, pp. 8855–8862.

[31] F. Veiga, R. Akrour, and J. Peters, "Hierarchical tactile-based control decomposition of dexterous in-hand manipulation tasks," *Front. Robot. AI*, vol. 7, 2020, Art. no. 521448.

[32] G. Khandate, C. P. Mehlman, X. Wei, and M. Ciocarlie, "Dexterous in-hand manipulation by guiding exploration with simple sub-skill controllers," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024, pp. 6551–6557.

[33] H. Qi, B. Yi, M. Lambeta, Y. Ma, R. Calandra, and J. Malik, "From simple to complex skills: The case of in-hand object reorientation," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2025, pp. 14291–14298.

[34] K. Shaw, A. Agarwal, and D. Pathak, "LEAP Hand: Low-cost, efficient, and anthropomorphic hand for robot learning," in *Proc. Robot.: Sci. Syst. XIX*, 2023.

[35] V. Makoviychuk et al., "Isaac Gym: High performance GPU-based physics simulation for robot learning," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks*, vol. 1, 2021.

[36] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[37] M. Quigley et al., "ROS: An open-source robot operating system," in *Proc. ICRA Workshop Open Source Softw.*, Kobe, Japan, 2009, Art. no. 5.